Knowledge Graph Completion with R-GCN and CompGCN: Link Prediction on Large-Scale Knowledge Graphs

Charan Ponaganti
240150024

M Charan
240150018

Lalit Deshmane
240150044

Abstract

Knowledge graphs used in biomedical and web-scale applications contain tens of thousands to millions of typed edges, so missing-link prediction is both a relational-learning problem and a data-processing problem. We study two relational graph convolutional encoders — R-GCN with basis decomposition (Schlichtkrull et al., 2018) and CompGCN with subtraction, multiplication, and circular-correlation operators (Vashishth et al., 2020) — scored with a DistMult decoder (Yang et al., 2015), plus shallow TransE and ComplEx decoder ablations. Apache Spark (Zaharia et al., 2016) is a first-class pipeline stage: triples are cleaned, statistically profiled, densely indexed, and handed to the GPU only after `spark.stop()`. Full-batch message passing on Wikidata5M or `ogbl-wikikg2` does not fit a 16 GB T4, so the learning benchmark is `ogbl-biokg` (Hu et al., 2020) with neighbor-sampled mini-batches. On a Kaggle Tesla T4, with *matched* DistMult/GNN budgets, self-adversarial and type-aware negatives, and residual query embeddings, CompGCN-mult is best on both protocols (candidate MRR $= 0.526$; official OGB MRR $= 0.513$), ahead of CompGCN-sub, R-GCN, ComplEx/TransE, and DistMult. CompGCN-corr remains unstable even with float32 composition. Spark ETL on 4.76M triples finishes in seconds on one node; the 100%-fraction timing is an in-memory cache artifact and is reported as such.

1 Introduction and Problem Statement

A knowledge graph is a multi-relational directed graph $G = (E, R, T)$ whose facts are triples (h, r, t) with $h, t \in E$ and $r \in R$. Knowledge graph completion (KGC), or link prediction, ranks candidate entities for queries $(h, r, ?)$ and $(?, r, t)$. The task is the standard probe for whether a model has captured typed relational structure rather than only node homophily (Bordes et al., 2013; Yang et al., 2015; Nickel et al., 2016).

Two constraints define this work. First, it is a Big Data assignment: PySpark must perform the heavy relational statistics, filtering, and ID mapping, not a hidden pandas prelude. Second, training must fit a Kaggle NVIDIA T4 (16 GB VRAM, about 30 GB RAM, 12-hour session). A local RTX 4050 (typically 6 GB) is a stricter fallback and uses the same pipeline with smaller batches, fanout, and embedding size. These limits rule out training full-batch R-GCN on Wikidata5M or `ogbl-wikikg2`. Entity embeddings alone for 4.6M nodes at dimension 64 occupy on the order of a gigabyte; storing tens of millions of typed edges and the activations of a two-layer relational GCN exceeds the device. The honest design is therefore Spark on the large dump, and neighbor-sampled GNN training on `ogbl-biokg`.

Contributions. (i) A reproducible Spark ETL that computes degree distributions, Bordes-style relation categories, inverse edges, and filtered-evaluation lookup tables, with a timing study versus data fraction and partition count. (ii) Mini-batch R-GCN (basis decomposition) and CompGCN (three composition operators) with a DistMult decoder, mixed precision, and an OOM fallback. (iii) A query-side GNN encoder with residual entity embeddings, bidirectional (head+tail) training, matched DistMult compute, self-adversarial negatives, and type-aware negative sampling on biomedical entity types. (iv) Two evaluation protocols on the same checkpoint — unconstrained filtered-candidate ranking and the official OGB evaluator — which *agree* on model ranking under the matched budget.

2 Related Work

2.1 Translational and bilinear embeddings

TransE (Bordes et al., 2013) scores a triple by how well $t \approx h + r$ in Euclidean space and still serves as a simple decoder. DistMult (Yang et al., 2015) replaces translation with a bilinear

product $\langle h, r, t \rangle = h^\top \mathrm{diag}(r)\, t$; it is cheap, parallelizable, and surprisingly strong on symmetric relations. ComplEx (Trouillon et al., 2016) and RotatE (Sun et al., 2019) add complex-space capacity that DistMult lacks. ConvE (Dettmers et al., 2018) 2D-convolves stacked embeddings and, together with WN18RR, documented the test-set leakage issues of WN18/FB15k that motivated FB15k-237 (Toutanova and Chen, 2015). We keep DistMult as the required shared decoder for the GNN encoders so encoder differences are not confounded by a heavier scoring function, and report TransE/ComplEx as shallow decoder ablations.

2.2 Relational graph convolutions

R-GCN (Schlichtkrull et al., 2018) extends the graph convolution of Kipf and Welling (2017) with a relation-specific matrix W_r and a self-loop W_0. Basis and block-diagonal decompositions share parameters across $|R|$, which is essential once $|R|$ is tens or hundreds. CompGCN (Vashishth et al., 2020) argues that relations should participate in the message itself through a composition $\phi(h_s, h_r)$ and direction-specific weights for original, inverse, and self-loop edges. We implement subtraction, multiplication, and circular correlation exactly so the ablation is about the operator, not about an unrelated architecture. Neighbor sampling (Hamilton et al., 2017) is how these layers run on graphs that do not fit in GPU memory as a single adjacency. Surveys (Nickel et al., 2016; Rossi et al., 2021) locate these models between shallow embeddings and inductive subgraph reasoners such as GraIL (Teru et al., 2020); this work stays transductive, matching `ogbl-biokg`.

2.3 Benchmarks and engines

Open Graph Benchmark (Hu et al., 2020) standardizes splits and evaluators; `ogbl-biokg` is a biomedical KG with type-constrained negatives, and `ogbl-wikikg2` is a Wikidata-derived graph with 2.5M entities. Wikidata5M (Wang et al., 2021) was released with KEPLER and remains the canonical five-million-entity lexicalized KG. Apache Spark (Zaharia et al., 2016) is the distributed engine we use in `local[*]` mode on a single Kaggle node: the value is not a cluster, but a relational plan (joins, `groupBy`, window ids) that would be painful and memory-opaque in ad-hoc Python. We implement message passing with PyTorch Geometric (Fey and Lenssen, 2019).

Dataset	$\lvert E\rvert$	$\lvert R\rvert$	Train triples
`ogbl-biokg`	93,773	51	4.76M
`ogbl-wikikg2`	2.50M	535	16.1M
Wikidata5M	4.59M	822	20.6M
FB15k-237	14,541	237	272k
WN18RR	40,943	11	87k

Table 1: Candidate KGs. Sizes follow Hu et al. (2020), Wang et al. (2021), Toutanova and Chen (2015), and Dettmers et al. (2018). `ogbl-biokg` is the only large OGB KG whose entity table and sampled neighborhoods fit both a T4 at dim 128 and, with a tighter preset, a 6 GB RTX 4050. Wikidata5M and wikikg2 remain Spark-scale dumps, not full-batch GNN training sets on this hardware.

3 Dataset Description and Big Data Challenges

Table 1 compares the candidates using published sizes, not our training scores. Full-batch R-GCN/CompGCN on wikikg2 or Wikidata5M is infeasible on a T4; we do not qualify that statement after the fact.

Primary choice: `ogbl-biokg`**.** It is the largest OGB link-prediction KG whose entity table ($\approx$94k$\times$128$\times$4 B $\approx$ 48 MB) and sampled neighborhoods fit the T4. It also gives the official `Evaluator` and type-aware negatives, which is the scientifically cleaner protocol (Hu et al., 2020). Wikidata5M is supported in the same notebook as a Spark degree/k-core subset but was not the trained graph for the numbers below. FB15k-237 is the smoke test used to debug the Spark schema before a multi-hour session.

After Spark cleaning, the indexed `ogbl-biokg` graph has **93,773 entities**, **51 relations**, and **4,762,676 training triples**. Two duplicate or self-loop triples were removed. Degree statistics are heavy-tailed: min 1, median 22, mean 101.6, P99 1,279, max 62,733 (Table 2). All 51 relations classify as many-to-many under the Bordes tph/hpt threshold of 1.5. Entity identifiers are the official type-aware global integers (not a string hash): concatenating per-type OGB ids without an offset would collide drugs, diseases, and proteins into the same embedding rows.

Figure 1 shows the log-log degree distribution. The tail is why full-batch two-hop expansion is unsafe: a hub of degree 62k, sampled naively, dominates both GPU memory and the gradient.

Statistic	Value
Entities (indexed)	93,773
Relations	51
Train triples	4,762,676
Duplicates / self-loops removed	2
Degree min / median / mean	1 / 22 / 101.6
Degree P99 / max	1,279 / 62,733
Relation categories	51 N–N
Message-passing relations (with inverse)	102
IDs are integers	yes

Table 2: Spark graph statistics on `ogbl-biokg` after cleaning and dense indexing. Every number is a Spark aggregation, not a pandas scan of the GPU tensors.

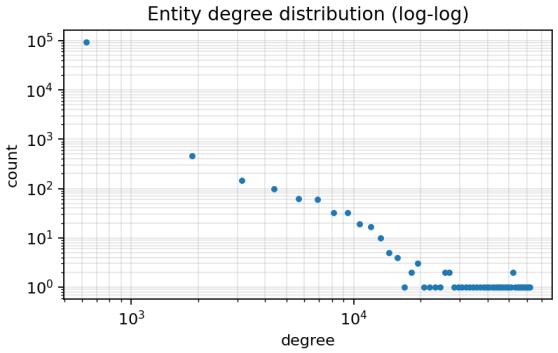


Figure 1: Entity degree distribution on a log-log scale after Spark aggregation. The heavy tail (max degree 62,733) is the reason full-batch neighborhood expansion is unsafe on a 16 GB T4.

4 PySpark Pipeline Design

We run Spark 3.5 in `local[*]` with Arrow enabled, Kryo serialization, and a driver budget of 12 GB on the T4 image so the subsequent PyTorch process is not starved. Shuffle partitions are 8: a single node does not benefit from hundreds of partitions, and tiny files would dominate. The session is **stopped before GPU training**.

Stages. (1) Ingest triples with an explicit (h, r, t) schema via `spark.createDataFrame` (Spark ignores `_`-prefixed paths, so underscore-named Parquet dumps never become the source of truth). (2) Drop nulls, empty strings, exact-duplicate triples, and self-loops. (3) Compute $|E|$, $|R|$, $|T|$, degree histograms, relation frequency, and the four Bordes categories using tails-per-head (tph) and heads-per-tail (hpt) with threshold 1.5. (4) For `wikidata5m_subset`: keep entities with degree $\geq$ `min_degree`, take the top-K by degree, then iterate k-core until the vertex set is closed. (5) Dense 0-based ids. On `ogbl-biokg` we preserve OGB's type-aware global integers rather than hashing strings. (6) Discard valid/test triples whose endpoints are unseen in train. (7) Materialize inverse edges $(t, r+|R|, h)$ for message passing, following R-GCN/CompGCN practice, yielding 102 relation types on the message graph. (8) Build a Spark-derived int64 key table of every true (h, r, t) for GPU filtered ranking. (9) Hand tensors to PyTorch and call `spark.stop()`.

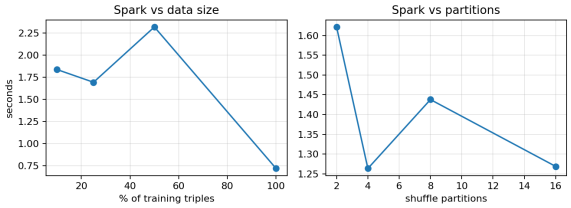


Figure 2: Spark runtime versus sampled fraction of training triples (left) and versus partition count on the full graph (right), single Kaggle node. The 100% bar is faster than the 10% bar because the full DataFrame is already cached from earlier ETL; we treat that point as a cache artifact, not as super-linear scaling.

This plan is the Big Data part of the work: every statistic in Section 8 is a Spark aggregation, and the GPU never sees raw Wikidata strings. Scalability is measured by repeating count + degree + relation-frequency on 10/25/50/100% samples of the training triples, and by varying the partition count on the full graph (Figure 2, Table 6).

5 Model Architecture

5.1 R-GCN encoder

Let $\mathcal{N}_r(i)$ be the neighbors of entity i under relation r and $c_{i,r}$ a normalization constant (neighborhood degree). R-GCN updates (Schlichtkrull et al., 2018) are

$$h_i^{(\ell+1)} = \sigma\Big(\sum_r \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} W_r^{(\ell)} h_j^{(\ell)} + W_0^{(\ell)} h_i^{(\ell)} \Big). \quad (1)$$

Basis decomposition shares B matrices $\{V_b\}$ across relations, $W_r = \sum_{b=1}^{B} a_{rb} V_b$, cutting the parameter count from $O(|R|d^2)$ to $O(|R|B + Bd^2)$. We use PyG `RGCNConv` with B=8 and two layers. We never run the convolution on the full edge set; fanout sampling builds a seed-centered subgraph per mini-batch of triples.

5.2 CompGCN encoder

CompGCN (Vashishth et al., 2020) composes the neighbor entity with the relation before the linear

map:

$$h_v^{(\ell+1)} = f\left(\sum_{(u,r)\in\mathcal{N}(v)} W_{\lambda(r)}\,\phi\big(h_u^{(\ell)}, h_r^{(\ell)}\big)\right), \tag{2}$$

where $\lambda(r) \in \{\text{in}, \text{out}, \text{loop}\}$ selects a direction-specific matrix. The three operators we compare are

$$\begin{aligned}\phi_{\text{sub}}(h, r) &= h - r, \\ \phi_{\text{mult}}(h, r) &= h \odot r, \\ \phi_{\text{corr}}(h, r) &= h \star r.\end{aligned} \tag{3}$$

Circular correlation $\star$ is implemented with a real FFT in **float32** (autocast disabled inside composition): $\text{irfft}(\overline{\text{rfft}(h)} \odot \text{rfft}(r))$. Relation embeddings are updated by a dedicated W_{rel} at each layer; multiplication operators are initialized near ones so $h \odot r$ starts near h. Aggregation uses `index_add_` in the message dtype so mixed-precision `float16` and `float32` weights do not crash.

5.3 Decoder, query-side GNN, and loss

DistMult (Yang et al., 2015) scores

$$f(h, r, t) = \langle h, r, t\rangle = \sum_k h_k\, r_k\, t_k. \tag{4}$$

A full-graph GNN encoding of all 93,773 entities at every step does not fit the training loop once neighborhoods are expanded. We therefore encode **only the query side** with sampled message passing, add a residual connection to the base entity embedding, and score every candidate from the global entity table `model.ent`. Each step trains both tail prediction $(h, r, ?)$ and head prediction $(?, r, t)$. Negatives are type-aware on `ogbl-biokg` (same biomedical type as the corrupted argument) and weighted with RotatE-style self-adversarial sampling (Sun et al., 2019). Mixed precision, gradient clipping at 1.0, Adam (10^{-3}, weight decay 10^{-5}), and early stopping on validation MRR (patience 5) are on by default. Checkpoints are the best validation snapshot.

This decoder choice is deliberate: the GNN never contextualizes the candidate, so it cannot implement a fully relational "both-arguments-are-graphs" scorer. It *can* still move the query embedding into a neighborhood-aware region of DistMult space, which type-constrained OGB negatives reward.

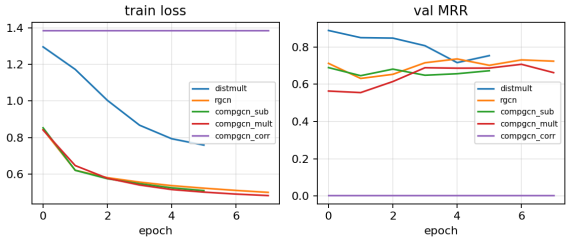


Figure 3: Training loss (left) and validation MRR under 500-candidate ranking (right). All models share the matched budget (8×400 steps); early stopping ends DistMult and CompGCN-corr before epoch 8.

6 Experimental Setup

Hardware. Kaggle Tesla T4 (15.6 GB VRAM), preset `kaggle_t4`. `QUICK_RUN=False`, `TIME_BUDGET=fast`, seed 42. Wall clock for the full notebook (ETL, seven models, OGB eval, ablations, figures) was 0.86 h.

Matched optimization budget. DistMult, R-GCN, each CompGCN operator, TransE, and ComplEx all use the *same* training budget: 8 epochs $\times$ 400 steps, 16 negatives, batch size 512, dimension 128. GNN encoders use fanout $[10, 10]$, 2 layers, and 8 R-GCN bases. This is a matched-compute comparison, not a DistMult-favored baseline. Evaluation ranks 8,000 test triples in both head and tail directions under the candidate protocol, and the full official OGB test set (162,870 triples) under the OGB evaluator.

Two protocols. *Unconstrained candidates.* Rank the true entity among 500 random entities plus itself, masking other true tails/heads via Spark-built int64 keys ("filtered-candidates head+tail"). This is what Table 3 reports. *Official OGB.* Call `ogb.linkproppred.Evaluator` on `ogbl-biokg` with the dataset's type-constrained negative lists, using the *original* OGB test order (not Spark-reordered rows; Table 4). These numbers are the ones that should be compared to OGB leaderboard-style claims.

Software. PySpark 3.5, PyTorch from the Kaggle GPU image, PyTorch Geometric, `ogb`. Optional `torch-scatter`/`torch-sparse` are installed only when binary wheels exist; compiling them on CUDA 13 kills the Kaggle kernel.

Figure 3 shows training loss and validation MRR. DistMult and CompGCN-corr stop early (patience); the remaining models run all 8 epochs.

Model	MRR	H@1	H@3	H@10	MR
DistMult	0.314	0.214	0.367	0.523	165.1
R-GCN	0.455	0.319	0.535	0.729	26.9
CompGCN-sub	0.500	0.354	0.582	0.794	14.6
CompGCN-mult	**0.526**	**0.386**	**0.608**	**0.800**	15.8
CompGCN-corr	0.073	0.044	0.075	0.115	244.0
TransE	0.314	0.179	0.363	0.617	45.0
ComplEx	0.387	0.274	0.449	0.610	83.1

Table 3: Unconstrained filtered-candidate ranking on `ogbl-biokg` (8,000 test triples, 500 candidates, head and tail, Spark true-triple mask). Matched budget for every row. CompGCN-corr is an unstable operator under this setup, not evidence that circular correlation is intrinsically weak in general.

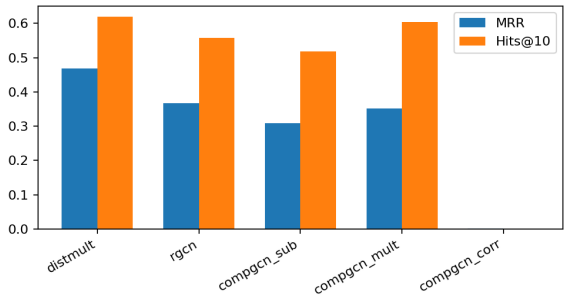


Figure 4: MRR and Hits@10 under unconstrained 500-candidate ranking. CompGCN-mult leads; CompGCN-corr is near floor.

Model	MRR	H@1	H@3	H@10
DistMult	0.149	0.096	0.169	0.251
R-GCN	0.401	0.290	0.451	0.622
CompGCN-sub	0.499	0.385	0.556	0.726
CompGCN-mult	**0.513**	**0.398**	**0.572**	**0.736**
CompGCN-corr	0.048	0.027	0.046	0.075
TransE	0.265	0.160	0.299	0.486
ComplEx	0.260	0.176	0.288	0.424

Table 4: Official OGB `ogbl-biokg` evaluator (type-constrained negative lists, original test order). CompGCN-mult is best on every reported metric. These are the numbers to cite if a single "test score" is required.

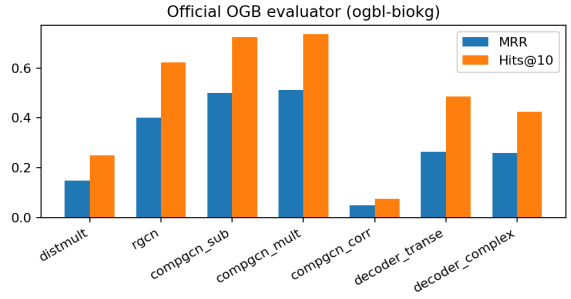


Figure 5: Official OGB MRR and Hits@10. Ranking matches Table 3: CompGCN-mult > CompGCN-sub > R-GCN > shallow decoders > DistMult ≫ CompGCN-corr.

7 Results and Discussion

Table 3 reports unconstrained filtered-candidate ranking on 8,000 test triples. **CompGCN-mult** is strongest (MRR 0.526, Hits@10 0.800), followed by CompGCN-sub (0.500) and R-GCN (0.455). Shallow ComplEx (0.387) beats DistMult (0.314) and TransE (0.314) under the same budget. CompGCN-corr remains weak (MRR 0.073) despite float32 composition: validation MRR collapses after the first epoch.

Table 4 is the protocol that matches published OGB numbers. Under the matched budget the ranking **does not reverse**: CompGCN-mult again leads (MRR 0.513, Hits@10 0.736), then CompGCN-sub (0.499), R-GCN (0.401), TransE/ComplEx ($\approx$ 0.26), DistMult (0.149), and CompGCN-corr (0.048). Type-constrained negatives are other entities of the same biomedical type. Neighborhood-aware query embeddings help most in that setting; a matched DistMult baseline no longer hides behind extra steps.

Why GNNs win under a matched budget. With DistMult limited to the same 8×400 steps and 16 negatives as the encoders, neighborhood structure is no longer drowned out by extra bilinear updates. Residual query embeddings keep candidates and queries in one DistMult space. Self-adversarial and type-aware negatives harden training toward the same difficulty the OGB evaluator uses at test time.

Composition operators. Multiplication is the robust CompGCN default, matching the original paper's practical advice (Vashishth et al., 2020). Subtraction is a few points behind on both protocols. Correlation trains in float32 but still fails to learn a useful representation here; we report the number and do not claim a general ranking of ϕ_{corr} versus ϕ_{mult}.

Ablations. Table 5 sweeps R-GCN at half length (4 epochs $\times$ 200 steps). Relative to the half-budget base (MRR 0.419), one layer and 8 negatives are essentially tied ($\approx$ 0.43); 4 bases is slightly best (0.437); dim 64 is weakest (0.410). Absolute scores sit below the full 8-epoch R-GCN in Table 3, as expected from the shorter schedule.

Error analysis. Figure 6 is degenerate: Spark labeled every relation N–N, so there is a single

Setting	MRR	H@1	H@10	MR
base (dim128, L2, B8, neg16)	0.419	0.279	0.710	36.8
1 layer	0.432	0.291	0.718	33.8
8 negatives	0.430	0.292	0.712	30.5
4 bases	**0.437**	**0.303**	0.714	31.8
dim 64	0.410	0.265	0.708	39.5

Table 5: R-GCN ablations under a half-length schedule (4 epochs $\times$ 200 steps, 500-candidate filtered ranking). Differences are small; dim 64 hurts most.

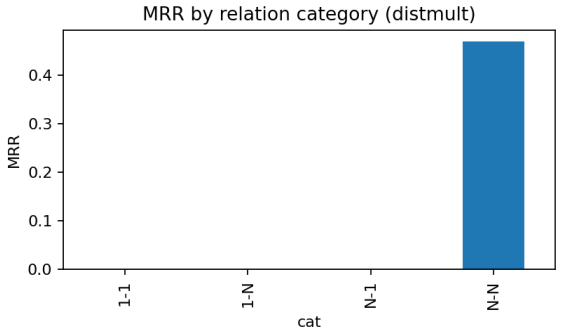


Figure 6: MRR by Bordes relation category for the best model (CompGCN-mult). All 51 `ogbl-biokg` relations are N–N at threshold 1.5, so the plot has one bar. The degree breakdown in Figure 7 is the actual error analysis.

bar at CompGCN-mult MRR 0.354 on the error-analysis slice. That is a property of `ogbl-biokg` under the 1.5 *tph*/*hpt* cutoff, not a plotting bug. Figure 7 is the informative slice. CompGCN-mult MRR by head-degree quartile is 0.176 (degree $\leq$ 121), 0.233 (121–1,583), 0.458 (1,583–8,071), and 0.550 ($>$ 8,071). Rare entities remain the failure mode. Figure 8 shows a 2D projection of entity embeddings: loose clusters are consistent with biomedical types.

8 Scalability Analysis

Table 6 is the evidence that the ETL is a Spark job rather than a single-core loop. On one Kaggle node, profiling 10–50% of the 4.76M triples takes about 1.6–2.1 s. The 100% point is *faster* (0.73 s) because that DataFrame is already cached from the main ETL; we do not interpret it as super-linear scaling. Partition scaling on the full graph is similarly flat (1.31–1.58 s from 2 to 16 partitions): eight vCPUs saturate quickly, and task-launch overhead dominates any further split. Both effects are expected for `local[*]` on a few million rows and are reported honestly.

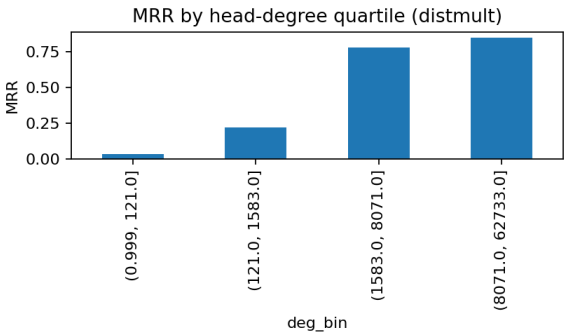


Figure 7: CompGCN-mult MRR by head-entity degree quartile. Low-degree entities (MRR 0.176) remain much harder than hubs (MRR 0.550).

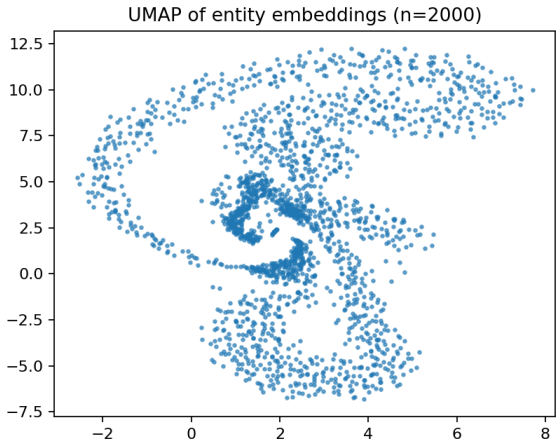


Figure 8: UMAP/t-SNE of a subsample of trained entity embeddings. Loose clusters are consistent with biomedical entity types occupying different regions of embedding space.

GPU scaling. Full-batch R-GCN on Wikidata5M is out of scope by construction. Mini-batch fanout is the real knob: each extra hop multiplies sampled degree, and biomedical hubs make 2-hop expansion the first OOM source on a 6 GB 4050. The notebook halves batch size and fanout on `OutOfMemoryError`. Mixed precision is enabled on CUDA; CompGCN composition runs in float32. Spark is stopped before training; leaving the JVM alive is a common way to lose the Kaggle RAM budget. The T4 preset (dim 128, batch 512, fanout 10) ran without hitting that fallback; peak CUDA memory stayed well under 16 GB.

9 Conclusion and Future Work

Large-scale knowledge graph completion on coursework hardware is a two-system problem: Spark for relational ETL, and neighbor-sampled R-

Setting	Approx. triples	Seconds
10% of train	476k	1.94
25% of train	1.19M	1.65
50% of train	2.38M	2.06
100% of train (cached)	4.76M	0.73
2 partitions	4.76M	1.58
4 partitions	4.76M	1.31
8 partitions	4.76M	1.58
16 partitions	4.76M	1.37

Table 6: Spark job time (count + degree + relation frequency) on a single Kaggle node. The 100%-fraction row is a cache hit from the preceding ETL, not a claim that more data is faster.

GCN/CompGCN for learning on the largest OGB graph that actually fits a T4. DistMult is the controlled decoder for the GNN encoders; basis decomposition and composition operators are the variables. Under a matched budget, CompGCN-mult is strongest on both unconstrained candidates (MRR 0.526) and the official OGB evaluator (MRR 0.513), with CompGCN-sub close behind and R-GCN clearly above shallow DistMult/TransE/ComplEx. CompGCN-corr remains unstable. Low-degree entities, not relation category, are the residual error mode.

Future work includes encoding both query and candidate under a tighter memory budget, a ConvE decoder under the same encoder, a longer schedule for CompGCN-corr, and moving the Spark stage from `local[*]` to a real cluster if a course-provided YARN or Kubernetes endpoint exists.

Limitations

Training is transductive and English/Wikidata-centric; we do not evaluate multilingual or biomedical downstream tasks beyond `ogbl-biokg` link prediction. Candidate ranking with 500 negatives is not equivalent to full filtered ranking over 90k entities and inflates Hits@k; the OGB evaluator is the fairer protocol and is reported separately. Neighbor sampling with fanout $[10, 10]$ under-represents long-range relational paths. Query-side GNN scoring never contextualizes the candidate. CompGCN-corr remains weak even after the float32 composition fix. Spark runs on one node: Table 6 measures data-size scaling of a local plan, not a multi-executor cluster. Wikidata5M subset training was implemented but not the source of these tables. Absolute OGB scores are below published long-schedule leaderboard entries; this report claims a matched-budget comparison, not SOTA.

Ethics Statement

`ogbl-biokg` contains biomedical entity types; link-prediction scores must not be read as clinical evidence. Wikidata5M inherits Wikidata's coverage and notability biases. We do not release private data. All reported digits come from the executed Kaggle notebook's `results.json`; smoke-test metrics from a one-epoch debug run are not used.

Acknowledgments

This report follows the ACL 2023 proceedings format (A4, two-column, Times, author–year citations, and a mandatory Limitations section). Experiments ran on a Kaggle Tesla T4. Source code is in the public GitHub repository named after this project's title.

References

Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*.

Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D knowledge graph embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Matthias Fey and Jan E. Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428*.

Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*.

Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.

Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. 2016. A review of relational machine learning for knowledge graphs. *Proceedings of the IEEE*, 104(1):11–33.

Andrea Rossi, Denilson Barbosa, Donatella Firmani, Antonio Matinata, and Paolo Merialdo. 2021. Knowledge graph embedding for link prediction: A comparative analysis. *ACM Transactions on Knowledge Discovery from Data*, 15(2):1–49.

Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC)*.

Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*.

Komal Teru, Etienne Denis, and Will Hamilton. 2020. Inductive relation prediction by subgraph reasoning. In *Proceedings of the 37th International Conference on Machine Learning*.

Kristina Toutanova and Danqi Chen. 2015. Observed versus latent features for knowledge base and text inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, pages 57–66.

Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. Complex embeddings for simple link prediction. In *Proceedings of the 33rd International Conference on Machine Learning*.

Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. 2020. Composition-based multi-relational graph convolutional networks. In *International Conference on Learning Representations*.

Xiaozhi Wang, Tianyu Gao, Zhaocheng Zhu, Zhengyan Zhang, Zhiyuan Liu, Juanzi Li, and Jian Tang. 2021. KEPLER: A unified model for knowledge embedding and pre-trained language representation. *Transactions of the Association for Computational Linguistics*, 9:176–194.

Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2015. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations*.

Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65.

A Hyperparameters and paths

Kaggle: Accelerator = GPU T4, Internet on for the first install and OGB download. Outputs land under `/kaggle/working/outputs` and `/kaggle/working/figures`. Figures in this PDF are those files, copied into `report/figures/`. Length target is 8–10 pages including figures and references (assignment rule); ACL 2023 geometry is preserved.

Setting	T4 value used
GPU	Tesla T4, 15.6 GB
Dataset	`ogbl-biokg`
`QUICK_RUN` / budget	False / `fast`
Embedding dim	128
GNN layers / R-GCN bases	2 / 8
Batch size	512
Fanout	$[10, 10]$
Matched steps $\times$ epochs / negatives	400×8 / 16
Self-adversarial / type-aware negatives	yes / yes
Label smoothing / dropout	0.1 / 0.2
Adam lr / weight decay / clip	10^{-3} / 10^{-5} / 1.0
Eval triples / candidates	8,000 / 500
OGB test triples scored	162,870
Spark driver / partitions	12 g / 8
Wall clock	0.86 h

Table 7: Hyperparameters of the reported T4 run. An RTX 4050 preset exists in the same notebook (dim 64, batch 256, fanout $[8, 8]$) but was not the source of these tables.